Check for updates

CPT: Pharmacometrics & Systems Pharmacology

TUTORIAL OPEN ACCESS

A Practical Tutorial on Physics-Informed Networks for Pharmacometrics and Quantitative Systems Pharmacology

Nazanin Ahmadi Daryakenari[1] | Mohammad Kohandel[2]

[1]Center for Biomedical Engineering, Brown University, Providence, Rhode Island, USA | [2]Department of Applied Mathematics, University of Waterloo, Waterloo, Ontario, Canada

Correspondence: Mohammad Kohandel (kohandel@uwaterloo.ca)

Received: 2 May 2026 | **Revised:** 31 July 2026 | **Accepted:** 27 August 2026

Keywords: gray-box modeling | inverse problems | Kolmogorov–Arnold networks | pharmacodynamics | pharmacokinetics | physics-informed neural networks | quantitative systems pharmacology

ABSTRACT

Inverse problems in pharmacometrics and quantitative systems pharmacology (QSP) often involve sparse, noisy data, limited measurable states, and complex dynamical systems. Traditional parameter estimation methods can struggle with ill-posed problems, stiffness, discontinuities, and gray-box scenarios where only part of the system dynamics is known or observable. This tutorial provides an end-to-end, reproducible workflow for applying physics-informed neural networks (PINNs) to PK/PD/QSP inverse problems and gray-box discovery tasks. We present three worked case studies: (1) constant parameter recovery in a three-state compartmental system, (2) gray-box discovery of an unknown right-hand-side function, and (3) inference of a time-varying chemotherapy efficacy function under partial observation. For each, we provide implementation guidance, validation checks, and practical advice on collocation design, feature expansion, optimizer selection, numerical precision, constraint enforcement, loss weighting, and residual-based attention. We also discuss when to consider newer architectures such as Kolmogorov–Arnold networks (KANs) as drop-in alternatives to standard multilayer perceptrons. All code and notebooks are provided through our `PhINs` library, short for Pharmacometrics-Informed Networks, and are publicly available at https://github.com/NazAhmadi/PhINs/.

1 | Introduction and Motivation

Quantitative modeling in drug development relies heavily on solving inverse problems: estimating model parameters from observed data while respecting underlying physiological mechanisms. Pharmacokinetic/pharmacodynamic (PK/PD) and quantitative systems pharmacology (QSP) models are essential tools for understanding dose–response relationships, optimizing dosing regimens, and predicting clinical outcomes [1]. However, these models often face challenges including sparse and noisy clinical data, limited observability of system states, time-dependent or multirate dynamics, model stiffness, parameter non-identifiability, and computational expense [2].

Traditional parameter estimation methods such as nonlinear least squares or maximum likelihood estimation often become ill-posed problems, where multiple parameter combinations fit the data equally well. These challenges can become even more severe when only a subset of system states is observable, or when the underlying dynamics are stiff, time-varying, or exhibit abrupt response changes. Bayesian approaches provide uncertainty quantification but can be computationally intensive for complex models. In practice, many pharmacological models are only partially known: core physiological mechanisms are well established, while certain interactions, such as adaptive resistance, time-varying feedback, nonlinear absorption effects, or coupling terms between components in a QSP model, remain uncertain or

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial-NoDerivs License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2026 The Author(s). *CPT: Pharmacometrics & Systems Pharmacology* published by Wiley Periodicals LLC on behalf of American Society for Clinical Pharmacology and Therapeutics.

difficult to model explicitly. This motivates a gray-box modeling paradigm, in which trusted mechanistic components are preserved while unknown terms are inferred from data.

Recently, physics-informed neural networks (PINNs) have emerged as a promising alternative that incorporates mechanistic knowledge directly into the training process [3, 4]. PINNs have been explored for learning latent pharmacodynamic relationships [5], reconstructing tissue or tumor responses under different growth assumptions such as Verhulst and Montroll-type models [6], performing parameter inference in tumor-growth models with dose-driven discontinuities [7], and supporting gray-box discovery in systems pharmacology [8, 9]. Within a physics-informed framework, missing interactions can be represented by neural function approximators whose outputs are constrained by the known system dynamics, allowing recovery of mathematically consistent hidden components.

In pharmacometrics, the first PINN-based framework was AI-Aristotle, which introduced a physics-informed strategy for gray-box identification by combining PINNs with symbolic regression to discover missing or unmodeled dynamical terms while preserving interpretability and biological plausibility [10]. Subsequent developments broadened this direction. In particular, Compartment Model Informed Neural Networks (CMINNs) extended the use of physics-informed learning to nonlinear drug behavior, abrupt response changes, and multi-dose treatment settings, while also enabling inference of time-varying or piecewise-constant parameters and fractional-order extensions [11, 12]. These later developments also highlighted the importance of sequential or interval-wise training when the dynamics contain abrupt spikes, dose-driven transitions, or regime changes that are difficult to capture with a single global training pass [11, 12]. Together, these studies show how classical compartment models can be augmented by physics-informed learning to recover anomalous absorption, trapping, escape, tolerance, persistence, or resistance-like behavior from data.

The applicability of PINNs is also expanding beyond classical PK/PD into broader biomedical and systems pharmacology applications [13]. Recent work has demonstrated the value of PINNs and related physics-informed methods in biomedical science and engineering [14], systems toxicology and drug safety, including hypernetwork-enhanced PINNs for cardiac safety assessment [15], and electrophysiological modeling of anti-arrhythmic drug effects [16]. These advances suggest that physics-informed learning can serve not only as an inverse solver but also as a flexible framework for combining mechanistic priors with sparse or high-dimensional biological observations.

More recently, physics-informed Kolmogorov–Arnold networks (PIKANs) have been proposed as structured alternatives to standard multilayer perceptron PINNs. KAN-based architectures introduce a different representation bias based on learnable univariate functions and may offer advantages in representation efficiency or robustness for certain inverse problems [8, 17]. In particular, Chebyshev-based PIKANs and related variants have shown promise for gray-box discovery and ODE-based inverse problems where optimization stability, noisy data, or sharp response changes are important considerations. Nevertheless, standard PINNs remain the more mature and widely adopted baseline, and in practice, it is useful to compare Multilayer Perceptron (MLP) and KAN representations under the same physics-informed training workflow.

In practice, successful PINN or PIKAN performance depends not only on the baseline formulation but also on implementation choices such as preprocessing and normalization, feature expansion, collocation design, adaptive or residual-based loss weighting, network architecture selection, constraint enforcement, optimizer choice, and numerical precision. This tutorial provides a practical guide to applying PINNs with optional PIKAN extensions to pharmacometric and QSP inverse problems using our `PhINs` library, short for Pharmacometrics-Informed Networks. We present three worked examples: constant parameter recovery in a three-state compartmental system, gray-box discovery of an unknown right-hand-side function, and inference of a time-varying chemotherapy efficacy function under partial observation.

Our core promise: we provide an end-to-end, reproducible, and implementation-focused workflow for building, training, validating, and interpreting physics-informed models for PK/PD/QSP inverse problems and gray-box discovery.

How to Read This Tutorial

This tutorial demonstrates how to use the `PhINs` library to build PINN- and KAN-based models for pharmacometric and QSP inverse problems. The workflow covers data preparation, residual construction, model configuration, training, and validation. The tutorial assumes familiarity with ODE-based PK/PD/QSP models and basic Python programming, but no advanced deep learning background.

2 | Prerequisites and Notation

Throughout, we use the notation summarized in Table 1. Here, t denotes time, $x(t)$ the state vector, θ constant model parameters, $\phi(t)$ a time-varying parameter or unknown function when needed, and $\hat{x}(t)$ the neural approximation of the state variables. We consider three related problem settings: inverse problems, in which unknown constant parameters are inferred from observations; gray-box problems, in which part of the governing dynamics is known and part is inferred indirectly from observed data; and time-varying inference problems, in which an unknown parameter or hidden efficacy factor evolves over time.

2.1 | Key Notation

Problem Classification

- **Forward problem**: Given parameters θ and initial conditions, predict $x(t)$.
- **Inverse problem**: Given observations $\{y_i\}$, estimate unknown parameters θ, hidden states, or both.
- **Gray-box problem**: Part of $f(x, \theta, t)$ is known mechanistically, while an unknown component is learned from data.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 1 | Notation used throughout the tutorial.

Symbol	Description
t	Time
$x(t)$	State variables
θ	Constant model parameters
$\phi(t)$	Time-varying parameter or unknown function
y_i	Observed data at time t_i
$\hat{x}(t)$	Neural approximation of the state variables
$f(x, \theta, t)$	Known mechanistic model
$r(t)$	Physics residual, typically defined as $r(t) = \frac{d}{dt}\hat{x}(t) - f\left(\hat{x}(t), \theta, t\right)$
$\mathcal{L}$	Total training loss
$\mathcal{L}_{\text{data}}, \mathcal{L}_{\text{phys}}, \mathcal{L}_{\text{IC}}$	Data, physics, and initial-condition loss terms
$\lambda_{\text{data}}, \lambda_{\text{phys}}, \lambda_{\text{IC}}$	Corresponding loss weights

3 | Background

3.1 | Mechanistic ODE Models and Their Variants

Mechanistic models in pharmacometrics and quantitative systems pharmacology (QSP) are commonly formulated as systems of ordinary differential equations (ODEs) that describe the temporal evolution of biological, pharmacokinetic, or pharmacodynamic states. In general, we write as follows:

$$\frac{dx(t)}{dt} = f(x(t), \theta, t), \tag{1}$$

where $x(t)$ denotes the state vector, θ denotes model parameters, and f represents the mechanistic dynamics. Depending on the application, the components of $x(t)$ may represent drug amounts, concentrations, biomarker levels, tumor burden, or interacting cellular populations.

A classical pharmacokinetic example is a one-compartment model with first-order absorption and elimination:

$$\frac{dA_g(t)}{dt} = -k_a A_g(t), \tag{2}$$

$$\frac{dA_c(t)}{dt} = k_a A_g(t) - \frac{CL}{V} A_c(t), \tag{3}$$

$$C(t) = \frac{A_c(t)}{V}, \tag{4}$$

where $A_g(t)$ and $A_c(t)$ denote drug amount in the absorption and central compartments, k_a is the absorption rate, CL is clearance, and V is the volume of distribution. For the initial conditions $A_g(0) = D$ and $A_c(0) = 0$, this model admits the closed-form solution

$$A_g(t) = De^{-k_a t}, \qquad A_c(t) = \frac{Dk_a}{k_a - \frac{CL}{V}}\left(e^{-\frac{CL}{V}t} - e^{-k_a t}\right), \qquad C(t) = \frac{A_c(t)}{V}. \tag{5}$$

This expression is valid whenever $k_a \neq CL/V$ (In the degenerate case $k_a = CL/V$, the solution instead reduces to $A_c(t) = Dk_a t\, e^{-k_a t}$).

A representative pharmacodynamic example is a tumor-growth inhibition model:

$$\frac{dT}{dt} = k_g T\left(1 - \frac{T}{T_{\max}}\right) - E(t)T, \tag{6}$$

where $T(t)$ denotes tumor burden, k_g is the intrinsic growth rate, $T_{\max}$ is a carrying capacity, and $E(t)$ represents the drug effect. More generally, pharmacodynamic models may include direct-effect, indirect-response, or effect-compartment formulations, depending on the biological mechanism being represented.

Mechanistic Modeling Scenarios Covered in This Tutorial

- **Constant-parameter inverse problems**: The governing equations are known, and unknown fixed parameters are inferred from observations.
- **Gray-box discovery problems**: One right-hand-side term is unknown and is learned as a function of time or state variables.
- **Time-varying inference problems**: An unknown hidden efficacy or forcing factor evolves over time and must be inferred under mechanistic constraints.

3.2 | Identifiability Analysis

A central task in pharmacometrics and QSP is solving *inverse problems*: estimating unknown parameters, hidden states, or missing dynamical terms from observed data while preserving mechanistic consistency. In the simplest setting, the unknowns are constant parameters; in more difficult settings, only part of the state vector is observed, or the unknown itself is a time-varying function. These challenges are common in pharmacometric and QSP applications, where measurements are often sparse, noisy, and incomplete.

Identifiability is therefore essential. A parameter is *structurally identifiable* if it can be uniquely determined from ideal, noise-free observations, whereas *practical identifiability* reflects whether it can be estimated with acceptable uncertainty under realistic noise and limited observability [18]. In gray-box settings, these difficulties are amplified because multiple hidden functions or parameter combinations may explain the same observed trajectory unless the model is sufficiently constrained.

Traditional approaches to inverse problems include nonlinear least squares:

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

$$\hat{\theta} = \arg\min_{\theta} \sum_{i} \left(y_i - \hat{y}_i(\theta)\right)^2,$$

where θ denotes the candidate parameter vector, $\hat{\theta}$ is the estimated parameter vector that minimizes the objective function, y_i is the observed data point, and $\hat{y}_i(\theta)$ is the model-predicted observation at the same time or condition for a given parameter value. These methods remain foundational, but they can become computationally or statistically challenging when the problem is ill-posed, when multiple parameter combinations yield similar outputs, or when the unknown quantity is itself a function rather than a finite-dimensional parameter. In nonlinear dynamical systems, the likelihood surface may exhibit extended ridges, parameter compensation, or strongly correlated directions, so apparently good fits may still correspond to poorly identifiable parameterizations [19, 20].

Several complementary tools are used to assess identifiability. Local sensitivity-based measures such as the Fisher Information Matrix (FIM) approximate uncertainty through curvature near the optimum, but may be misleading when the objective function is highly nonlinear or poorly conditioned [19]. Bootstrap or Monte Carlo variance estimates can also fail to detect near-non-identifiable manifolds, particularly in settings where parameters and functions are inferred simultaneously [21, 22]. More global approaches such as profile likelihood provide a more robust assessment by systematically exploring the objective surface through repeated re-optimization and revealing finite confidence intervals, flat directions, or functional dependencies among parameters [18, 19]. In a recent systems biology study, practical identifiability was evaluated using two complementary methods: local Fisher Information Matrix (FIM) analysis and global profile likelihood [23]. This local–global combination is particularly important in nonlinear ODE models, where curvature-based measures alone may be misleading and profile likelihood can reveal weakly constrained directions that are otherwise missed. Based on the agreement between these two assessments, the search space of unknown parameters was restricted to ranges that support unique and identifiable parameter estimates [23].

In the context of this tutorial, identifiability should be viewed as a prerequisite for trustworthy physics-informed inference. Even when a PINN or PIKAN fits the observed data well, the recovered parameters or hidden functions may not be unique unless the inverse problem is sufficiently constrained. PINNs can help regularize training by enforcing mechanistic consistency, but they do not eliminate fundamental limitations caused by non-identifiability, sparse or noisy data, insufficient observability, or model misspecification. They should therefore be viewed as complementary tools rather than replacements for established frequentist, Bayesian, and mechanistic modeling approaches. Recent work shows that identifiability analysis before physics-informed inference can be used to restrict the parameter search range to identifiable regions, thereby improving recoverability and making the inverse problem better constrained [23]. For this reason, parameter and hidden-function recovery should be interpreted together with residual diagnostics, uncertainty analysis, biological plausibility, and, whenever possible, structural or practical identifiability assessment.

4 | Physics-Informed Learning: PINNs and PIKANs

4.1 | Physics-Informed Network Formulation

Physics-informed networks (PINNs) approximate the solution of a differential equation using a trainable function that is fitted not only to observed data but also to the governing dynamics [3]. For a system of the form:

$$\frac{dx}{dt} = f(x, \theta, t), \qquad x(0) = x_0, \tag{7}$$

the state trajectory is represented by a neural approximation.

$$x(t) \approx \hat{x}(t; w), \tag{8}$$

where w denotes the trainable network weights. The governing equations are enforced through the physics residual.

$$r(t) = \frac{d}{dt}\hat{x}(t) - f\left(\hat{x}(t), \theta, t\right), \tag{9}$$

where the derivative of $\hat{x}(t)$ with respect to time is computed by automatic differentiation.

Training is performed by minimizing a composite loss function that combines data fit, initial conditions, and physics enforcement:

$$\mathcal{L} = \lambda_{\text{data}}\mathcal{L}_{\text{data}} + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}} + \lambda_{\text{IC}}\mathcal{L}_{\text{IC}}, \tag{10}$$

with

$$\mathcal{L}_{\text{data}} = \frac{1}{N}\sum_{i=1}^{N}\left\|y_i - \hat{x}\left(t_i\right)\right\|^2, \tag{11}$$

$$\mathcal{L}_{\text{phys}} = \frac{1}{M}\sum_{j=1}^{M}\left\|r\left(\tau_j\right)\right\|^2, \tag{12}$$

$$\mathcal{L}_{\text{IC}} = \left\|\hat{x}(0) - x_0\right\|^2, \tag{13}$$

where $\left\{t_i\right\}_{i=1}^{N}$ are observation times and $\left\{\tau_j\right\}_{j=1}^{M}$ are collocation points used to evaluate the residual.

Figure 1 summarizes this physics-informed training structure. A neural representation, either an MLP or KAN, maps the input time t to the predicted state $\hat{x}(t)$. Automatic differentiation is used to compute $d\hat{x}/dt$, which enters the physics residual together with the governing ODE model.

This formulation is flexible enough to handle all three problem types considered in this tutorial. In a standard inverse problem, the unknowns are constant parameters optimized jointly with the network. In a gray-box problem, an unknown right-hand-side term can be represented as an additional network output. In a time-varying inference problem, the network can also represent an unknown function $\phi(t)$, such as a hidden efficacy or forcing factor.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

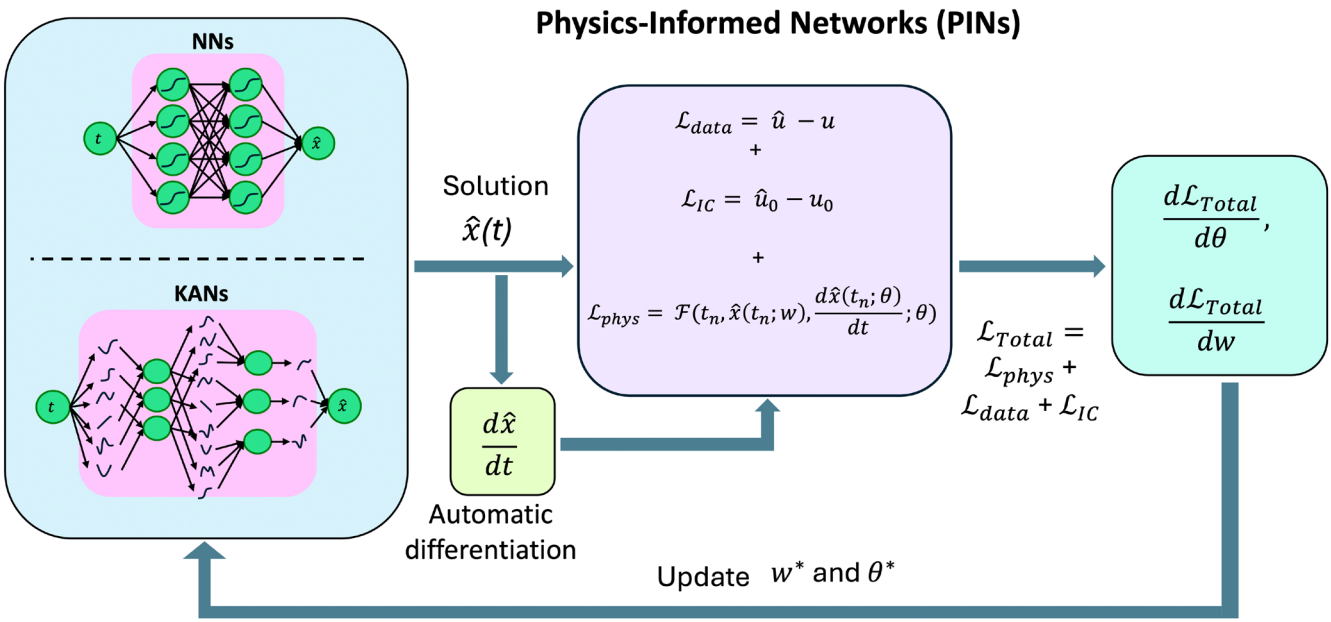


FIGURE 1 | Physics-informed network architecture. A neural representation, either an MLP or a KAN, maps the input time t to the predicted state $\hat{x}(t)$. Automatic differentiation computes the time derivative $d\hat{x}/dt$, which is used to construct the physics residual. Training minimizes a composite loss that combines data mismatch, initial-condition error, and physics-residual terms. Gradients of the total loss with respect to both neural-network weights and unknown model parameters are then used to update the model during optimization.

4.2 | Representation Models: MLPs Versus KANs

The physics-informed formulation above does not prescribe a unique neural representation for $\hat{x}(t)$. In this tutorial, we compare two alternatives: a standard multilayer perceptron (MLP) and a Chebyshev Kolmogorov–Arnold network (cKAN). Both are trained with the same composite physics-informed loss; only the functional representation differs.

4.2.1 | Multilayer Perceptron (MLP)

The baseline PINN uses a fully connected feed-forward neural network with tanh activation functions. For an input t, the forward pass through an L-layer MLP is as follows:

$$z^{(0)} = t, \tag{14}$$

$$z^{(\ell)} = \tanh\left(W^{(\ell)} z^{(\ell-1)} + b^{(\ell)}\right), \qquad \ell = 1, \ldots, L, \tag{15}$$

$$\hat{x}(t) = W^{(L+1)} z^{(L)} + b^{(L+1)}, \tag{16}$$

where $W^{(\ell)}$ and $b^{(\ell)}$ are the weight matrices and bias vectors. The final layer is linear, allowing the network to represent states as well as auxiliary outputs such as unknown functions or time-varying parameters.

The MLP serves as the default baseline in this tutorial because it is widely used, easy to implement, and often sufficient for many inverse ODE problems. We do not use ReLU for the PINN examples because the physics residual requires derivatives of the neural-network output with respect to time. Smooth activation functions such as tanh are preferred because their derivatives are continuous, which makes automatic-differentiation-based residual calculations more stable. In contrast, ReLU has piecewise-constant first derivatives, zero second derivatives almost everywhere, and a nondifferentiable point at zero. This derivative structure can make ReLU poorly suited for PINN training, especially when smooth ODE residuals or high-order derivatives are required.

4.2.2 | Chebyshev-KAN Representation

To investigate whether a more structured representation improves inverse robustness, the MLP can be replaced by a Kolmogorov–Arnold network (KAN) [17]. KANs are motivated by the Kolmogorov–Arnold representation principle and replace standard affine transformations with compositions of learnable univariate nonlinear mappings.

In the Chebyshev-based variant [24, 25], these mappings are represented by Chebyshev polynomial expansions:

$$\psi(\zeta) = \sum_{n=0}^{D} c_n T_n(\zeta), \tag{17}$$

where $T_n(\zeta)$ denotes the n-th Chebyshev polynomial and c_n are trainable coefficients.

In this tutorial, when we refer to PIKANs we use the Chebyshev-based physics-informed KAN architecture, and in some settings its *tanh-cPIKAN* variant, in which nested tanh nonlinearities are introduced to improve numerical stability [11, 26]. This stabilization is especially useful in inverse ODE problems where high-degree polynomial interactions may otherwise amplify derivatives and lead to sharp curvature in the loss landscape.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

For an L-layer tanh-cPIKAN with $\sigma(x) = \tanh(x)$, the hidden representation can be written recursively as follows:

$$z^{(0)} = \sigma(t), \tag{18}$$

$$z^{(\ell)} = \sigma\left(\sum_{i=1}^{H_\ell} \sum_{n=0}^{D} c_{i,n}^{(\ell)} T_n\left(\sigma\left(z_i^{(\ell-1)}\right)\right)\right), \qquad \ell = 1, \ldots, L, \tag{19}$$

followed by a linear readout

$$\hat{x}(t) = W z^{(L)}. \tag{20}$$

Compared with the MLP, this representation introduces a different inductive bias and can improve robustness in some noisy or ill-conditioned inverse problems [11, 26]. However, it is typically more computationally demanding and may require more careful tuning.

Why include both MLP and PIKAN in this tutorial?

- **PINNs** provide a simple and strong baseline that is easy to train and interpret.
- **PIKANs** offer a structured alternative that may improve robustness and representation quality in inverse and gray-box problems, especially in high-noise data settings.
- Using both architectures within the same physics-informed workflow makes it possible to compare representation effects without changing the residual formulation or loss construction.

For most new problems, we recommend starting with an MLP baseline and considering cKANs as an optional refinement.

5 | A Practical Workflow for Pharmacometrics/QSP PINNs

Figure 2 summarizes the workflow used throughout this tutorial. Across all examples, the same sequence is followed: define the mechanistic model, generate or assemble observations, preprocess and scale the data, specify collocation points, construct the residual function, choose a neural representation, set loss weights and constraints, train the model, tune hyperparameters, and validate the recovered parameters or hidden functions. Before training, time, state variables, outputs, and parameters are commonly scaled or nondimensionalized to comparable numerical ranges to improve optimizer conditioning and stabilize automatic-differentiation-based residual calculations [3, 14, 27].

5.1 | Data, Observation Models, and Noise

Pharmacometric and QSP datasets are often sparse, noisy, and only partially observed. Depending on the measurement process, noise may be modeled as additive, proportional, or log-normal. In this tutorial, we use synthetic datasets with controlled noise so that recovery of unknown parameters and hidden functions can be evaluated against known ground truth.

In the constant-parameter and gray-box examples, synthetic observations are generated at 50 time points with low noise. In the hidden-efficacy example, only a single state is observed. These settings reflect common pharmacometric challenges, including limited sampling, partial observability, and the indirect inference of mechanistic quantities from measured trajectories.

In general, we recommend scaling inputs and outputs when variables differ substantially in magnitude, using custom observation losses when only a subset of states is measured, and matching the observation model to the assumed noise structure whenever possible.

5.2 | Collocation Design and Physics Enforcement

Collocation points $\{\tau_j\}$ are the locations where the governing residual is enforced. Their choice is central to physics-informed training: too few points can weaken physics enforcement, whereas too many may increase computational cost without proportional benefit [23].

In the examples presented here, collocation points are primarily chosen uniformly over the time domain. In the current `PhINs` implementation, collocation points can also be randomly resampled during training using a built-in collocation sampler, enabling adaptive coverage of the time domain without modifying the residual definition. This simple strategy is sufficient for the tutorial problems and keeps the setup transparent. The constant-parameter and gray-box examples use 300 collocation points, whereas the hidden-efficacy example uses fewer points for computational efficiency when residual-based attention (RBA) is enabled. RBA is a gradient-free adaptive weighting scheme for PINNs in which pointwise attention multipliers are updated during training from a decayed history of normalized residual magnitudes. These multipliers act as an attention mask on the residual loss, increasing the influence of collocation points or loss components that have remained difficult to satisfy without introducing an auxiliary network or requiring gradient-based updates of the weights [28]. More generally, collocation sets may be fixed, randomly resampled during training, or selected adaptively. In `PhINs`, this is controlled through a collocation configuration that allows either fixed grids or uniform random resampling at each training epoch. Uniform sampling is a useful baseline, but it can be insufficient when the solution contains sharp gradients, localized transients, or strongly varying regions [29]. This has motivated adaptive sampling strategies that modify residual-point placement during training.

A useful distinction is between *resampling*, in which the full collocation set is periodically redrawn according to a probability distribution p_α, and *incremental sampling*, in which training starts from an initial set and new collocation points are added adaptively over time [29]. In both cases, the aim is to allocate more training effort to regions where the current model is less reliable.

A commonly used adaptive criterion is residual-based sampling, where collocation points are preferentially placed in regions with large residual magnitude [29–34]. By concentrating points in regions that contribute most strongly to the physics residual, this strategy can improve the efficiency of residual

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

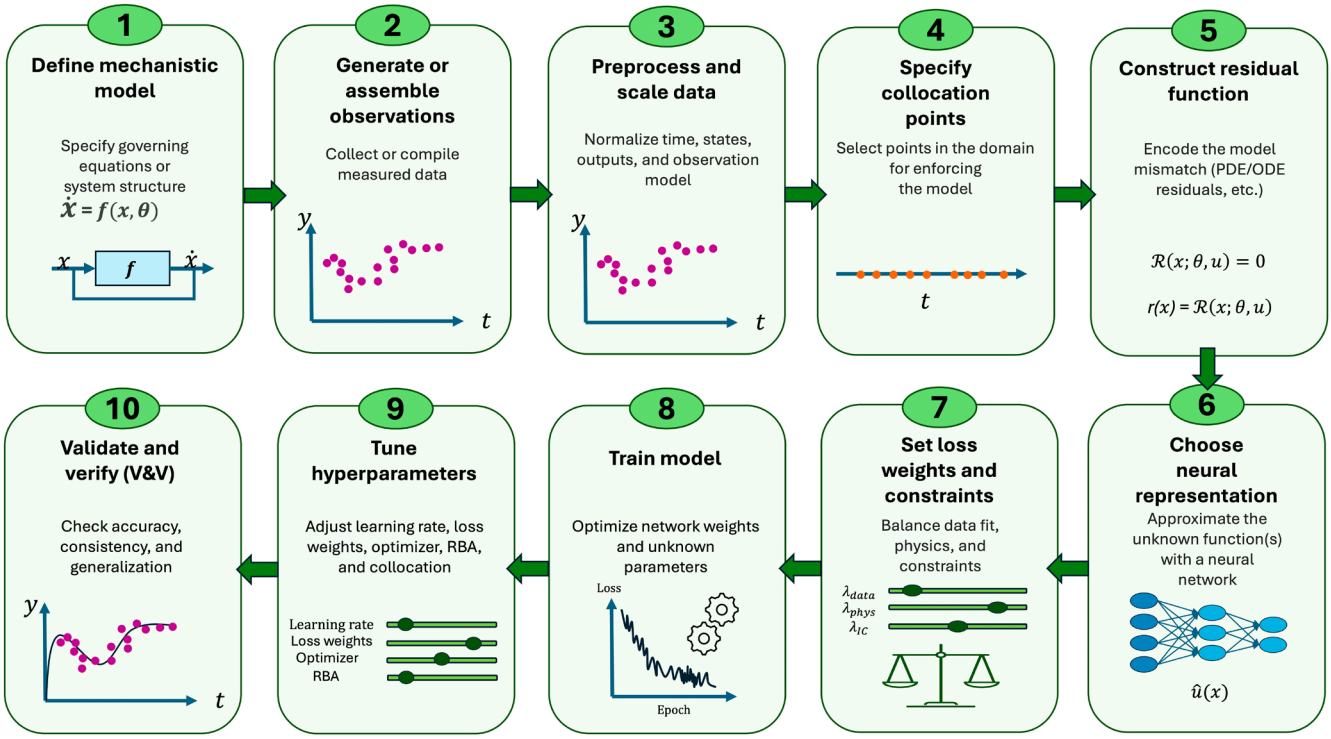


FIGURE 2 | Practical workflow for applying physics-informed neural networks to pharmacometrics and QSP inverse problems. The workflow begins by defining the mechanistic model and assembling observations, followed by preprocessing and scaling of time, states, outputs, and the observation model. Collocation points are then selected to specify where the governing equations are enforced, and the residual function is constructed from the mechanistic model using automatic differentiation. A neural representation, such as an MLP or KAN, is chosen to approximate the state variables and, when needed, unknown functions or parameters. Training then minimizes a composite objective that balances data fit, physics residual, initial condition, and constraint terms. Hyperparameters, including learning rate, loss weights, optimizer choice, residual-based attention, and collocation strategy, are tuned based on training diagnostics. The final step validates and verifies the recovered states, parameters, or hidden functions using trajectory agreement, residual diagnostics, and forward numerical simulation with the inferred quantities.

enforcement. However, adaptive residual-based sampling should be viewed primarily as an optimization heuristic and should not be interpreted as automatically beneficial. High residual regions do not always indicate insufficient collocation density. Persistent localized residuals may instead indicate model misspecification, missing mechanistic terms, unmodeled inputs, discontinuities, or inconsistency between the observation model and the data. In such cases, placing more points in high-residual regions may enforce the current model more strongly in those areas, which can be counterproductive if the model itself is misspecified. Residual diagnostics should therefore be interpreted alongside biological plausibility checks and, where possible, structural or practical identifiability analysis. A related alternative is failure-informed adaptive sampling, which defines a failure probability function and incrementally adds new collocation points in regions where this probability is high [35, 36]. Recent work also suggests that adaptive sampling and adaptive weighting are complementary rather than interchangeable: sampling changes *where* the residual is enforced, while weighting changes *how strongly* different points or terms influence the optimization [37].

For routine PK/PD and QSP inverse problems, we recommend starting with a fixed uniform collocation set because it is reproducible and easy to interpret. Random resampling or adaptive sampling can then be used when residual errors remain localized, the dynamics contain abrupt transitions, or fixed grids overemphasize specific regions of the time domain.

5.3 | Constraints, Scaling, and Parameterization

Mechanistic constraints are often essential for stable and interpretable inference. In the `PhINs` library, they can be imposed through parameter transforms such as softplus for positive constants, bounded mappings such as sigmoid-based transforms for time-varying functions, custom loss terms such as anchors or smoothness penalties, and prior search restrictions motivated by biological knowledge or identifiability analysis.

These considerations are especially important when learning hidden functions. In the third example, the unknown treatment-efficacy factor is small in magnitude, so direct unconstrained learning can yield poor recovery even when the observed trajectory is fitted well. To address this, the raw network output is mapped into a biologically plausible bounded range before entering the residual. This type of rescaling improves conditioning while preserving the mechanistic interpretation of the recovered quantity.

More generally, the same strategy is useful whenever a forcing term, transfer coefficient, or treatment-effect function is

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

small relative to the dominant state dynamics. In such cases, it is often better to learn a scaled latent quantity internally and map it back to physical units inside the residual. This is particularly helpful in inverse ODE problems where the state fit may appear accurate even though the hidden function itself is poorly recovered.

5.4 | Optimization, Precision, and Training Stability

Optimizer choice strongly influences PINN and PIKAN training. In practice, training still relies mainly on first-order methods such as stochastic gradient descent, Adam, and related adaptive-gradient variants because they are scalable, memory efficient, and relatively easy to tune. In this tutorial, we primarily use Adam or RAdam with cosine learning-rate decay. In the current `PhINs` release, the built-in optimizer options are `adam`, `adamw`, `radam`, `adamax`, `rmsprop`, and `sgd`, together with standard schedulers such as cosine, exponential, and piecewise decay.

At the same time, physics-informed optimization is often stiff, ill-conditioned, and highly nonconvex, so first-order training may plateau before the residual is adequately minimized. Although Newton-type methods can in principle accelerate convergence by exploiting curvature, direct Hessian-based optimization is usually too expensive for practical PINN training. This has motivated quasi-Newton and curvature-aware methods, including BFGS, L-BFGS, self-scaled BFGS, self-scaled Broyden, natural-gradient variants, and related second-order schemes that approximate or exploit curvature more efficiently. Recent studies show that these methods can substantially improve convergence and final accuracy on challenging PINN and PIKAN problems, especially after first-order stagnation [8, 38, 39].

Numerical precision is closely connected to this issue. Double precision can make a substantial difference when gradients are small, hidden functions are weakly constrained, the inverse problem is ill-conditioned, or second-order updates are used [8, 38]. In our own practice, it is especially helpful for inverse ODE problems involving weak hidden functions, sharp curvature, or high-accuracy recovery of time-varying quantities. A practical strategy is therefore to begin with a stable first-order optimizer, monitor whether training stalls, and then consider switching to a quasi-Newton or curvature-aware method when higher accuracy is needed. Beyond optimizer choice, several complementary strategies can improve convergence and stability, including adaptive loss weighting, warm-up schedules or two-stage training [10, 40], exponential feature scaling [11], and sequential or interval-wise training for problems with abrupt transients or piecewise changes [11, 12, 38]. These strategies do not replace careful optimizer selection, but they can improve conditioning and help resolve difficult dynamics more reliably. Several practical considerations are especially important:

- *Numerical precision and optimization*: double precision is often beneficial for ill-conditioned inverse problems, weakly constrained hidden functions, and curvature-aware or quasi-Newton updates [8, 38, 39].
- *Loss balancing and monitoring*: data, initial-condition, and physics residual terms should be monitored separately so that no single component dominates the optimization.
- *Representation choice*: MLPs provide a strong baseline, while KANs may improve robustness or representation quality in some inverse problems, particularly under higher noise levels [8, 23].
- *Future extensions*: curvature-aware optimizers remain promising additions for future versions of `PhINs`, especially when first-order methods plateau [26, 38, 39].

5.5 | Loss-Term Weighting and Residual Rebalancing

Physics-informed training combines several competing objectives, including data fit, initial conditions, residual enforcement, and sometimes auxiliary penalties or anchors. Their relative weights are rarely neutral: if the data term dominates, the model may fit observations while violating the governing equations; if the physics term dominates too early, optimization may stall before a useful state representation is learned. Loss weights should therefore be treated as modeling choices rather than fixed defaults.

A practical starting point is to assign explicit weights to each named loss term and monitor them separately during training. In our workflow, this includes data, initial-condition, and individual residual terms, as well as optional auxiliary terms such as anchors for hidden functions. In the original physics-informed framework, such global weights were typically fixed [3, 41]. More recent work has shown that dynamic global reweighting can substantially improve trainability. Examples include gradient-based annealing strategies based on back-propagated gradients [42], self-adaptive balancing methods [43, 44], causality-respecting or sequential weighting schemes for time-dependent problems [45, 46], and neural tangent kernel (NTK)-based adaptive weighting strategies that improve both trainability and predictive accuracy [47].

Beyond global weighting across loss terms, local weighting can be useful when certain collocation points are underrepresented in the total residual loss. This is especially important in multiscale or partially observed problems, where difficult regions may remain poorly resolved even as the overall loss decreases. To address this, local multipliers can be introduced so that different residual points contribute unequally to the optimization. Representative approaches include adversarial or differentiable self-adaptive reweighting [48, 49] and augmented-Lagrangian-style methods that enforce constraints through adaptive local penalties [50–52].

In this tutorial, we distinguish between adaptive loss-term weighting and RBA. Adaptive loss-term weighting changes the relative weights assigned to different components of the composite PINN objective, such as the data loss, initial-condition loss, physics-residual loss, or anchor loss. Its purpose is to rebalance competing objectives during training so that one term does not dominate the optimization. In contrast, RBA acts locally within a selected residual term. Thus, adaptive term weighting changes the balance *between* loss objectives, whereas RBA redistributes emphasis *within* the collocation set. Because these strategies serve different

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

roles, they may be combined when residual errors are localized or when the loss terms remain poorly balanced. In practice, we recommend starting with interpretable static weights, inspecting the separate loss histories, and enabling adaptive weighting or RBA only if the optimization remains poorly balanced.

5.6 | Partial Observation and Derived Observation Models

In many PK/PD and QSP problems, not all states are measured directly. For example, one may observe only plasma concentration, total tumor burden, or another aggregate quantity rather than the full state vector. In `PhINs`, this is handled by separating the mechanistic state model from the observation model: the residual enforces the full ODE system, while the data loss is applied only to the measured quantity. When all states are observed, the default data loss can be used directly. When only part of the system is observed, or when the measured output is a function of several states, a custom observation loss is more appropriate. For example, in monoclonal antibody pharmacokinetics, the measured total drug concentration may combine free drug and target-bound drug. If the mechanistic model includes latent free and bound concentrations, $C_{\text{free}}(t)$ and $C_{\text{bound}}(t)$, but the assay reports total concentration, then the observation model can be written as follows:

$$y(t) = C_{\text{free}}(t) + C_{\text{bound}}(t).$$

The data loss should therefore compare the measured total concentration with $\widehat{C}_{\text{free}}(t) + \widehat{C}_{\text{bound}}(t)$, while the residuals are still enforced separately for the free and bound state equations. This is a common setting in pharmacometrics. A model may contain several latent compartments while only one concentration is measured, or several latent tumor subpopulations while only total burden is observed. Physics-informed training is useful in such cases because the governing equations help regularize the unobserved states even when the data term acts on only one observable.

5.7 | Validation and Verification Strategy

A good fit to the observed data is not sufficient for trustworthy physics-informed inference. In V&V terminology, *verification* asks whether the equations are solved correctly, whereas *validation* asks whether the specified equations are appropriate for the system of interest. This distinction is important for PINNs because small data or residual losses do not necessarily imply that the mechanistic model is correct; model misspecification can still produce biased parameters, inaccurate hidden states, or poor generalization [53].

Verification should include residual diagnostics, training stability, collocation accuracy, and numerical consistency checks. Validation should include comparison with ground truth when available, recovery of hidden states or functions, biologically plausible parameter values, and forward simulation using the inferred quantities in a conventional ODE solver/PINNs [8, 11, 14]. Whenever possible, validation should also test held-out conditions, such as new dosing schedules, different initial conditions, or leave-one-out validation across subjects. For example, with 12 subjects, the model can be trained on 11 subjects and tested on the remaining one, repeated over all 12 folds, and summarized by the average prediction error.

6 | Worked Examples

The worked examples below illustrate how to use the `PhINs` library for inverse problems, gray-box discovery, and hidden time-varying function recovery. Each example follows the same workflow: import the library components, generate or load observations, create a `PINNDataBundle`, define the residual function, specify the model configuration, train with `PINNTrainer`, and validate the recovered quantities.

Main `PhINs` Components Used in the Worked Examples

`PINNConfig`	Top-level object that combines feature, architecture, training, and data settings.
`FeatureConfig`	Defines optional input feature expansions such as Fourier, exponential, or polynomial features. These transformations augment the raw time input before it enters the neural network and can improve representation efficiency or optimization conditioning when the expected solution has known structure. For example, exponential features can help represent compartmental PK trajectories that exhibit mono- or multi-exponential decay, while Fourier features can be useful for oscillatory or periodic dynamics.
`ArchitectureConfig`	Selects the neural representation, such as an MLP or Chebyshev-KAN.
`TrainingConfig`	Specifies epochs, optimizer, scheduler, loss weights, and optional RBA/adaptive weighting.
`DataConfig`	Defines input dimension, state names, and unknown parameter specifications.
`ParameterSpec`	Declares unknown quantities as constant parameters or time-varying network outputs.
`PINNDataBundle`	Stores observation times, observations, initial conditions, and collocation points.
`PINNProblem`	Combines the configuration, residual function, data bundle, and optional custom data loss.
`PINNTrainer`	Trains the model, applies optional collocation resampling, and returns predictions using `fit()` and `predict()`.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The basic imports used throughout the examples are as follows:

```
import jax
import jax.numpy as jnp
import numpy as np
from scipy.integrate import odeint

from phins import (
    PINNConfig, FeatureConfig,
    ArchitectureConfig,
    TrainingConfig, DataConfig,
    ParameterSpec,
    PINNDataBundle, PINNProblem,
    PINNTrainer,
    RBAConfig
)

jax.config.update("jax_enable_x64", True)
```

Double precision is enabled because inverse ODE problems and hidden-function recovery can be sensitive to small gradients and numerical precision.

6.1 | Example 1: Inverse Recovery of Constant Parameters

The first example uses a single-dose compartmental pharmacokinetic model with three states: $G(t)$ for the gastrointestinal absorption compartment, $B(t)$ for the central drug amount, and $U(t)$ for the eliminated drug amount. The pharmacokinetic model is as follows:

$$\frac{dG}{dt} = -K_G G, \tag{21}$$

$$\frac{dB}{dt} = K_G G - K_B B, \tag{22}$$

$$\frac{dU}{dt} = K_B B, \tag{23}$$

with

$$G(0) = 0.1, \qquad B(0) = 0, \qquad U(0) = 0.$$

The parameter K_G controls transfer from the gastrointestinal absorption compartment to the central compartment, while K_B controls elimination from the central compartment. In the synthetic data-generating model,

$$K_G^{\text{true}} = 0.72\,\text{h}^{-1}, \qquad K_B^{\text{true}} = 0.15\,\text{h}^{-1}.$$

During training, these parameters are treated as unknown and are inferred from noisy observations of all three states. Synthetic data are generated over 50 h with 2% and 7% noise, and 300 collocation points are used for residual enforcement. The purpose of this example is to demonstrate the basic `PhINs` workflow for constant-parameter recovery in a fully specified PK model.

6.1.1 | Creating the Data Bundle

For synthetic data, we first define the ODE right-hand side and solve it numerically. Here, `odeint` is used to generate the reference trajectories, after which measurement noise is added and the result is converted into the `PINNDataBundle` format.

```
def true_ode(y, t):
    G, B, U = y
    return [
        -KG_TRUE * G,
        KG_TRUE * G - KB_TRUE * B,
        KB_TRUE * B,
    ]

t_obs = np.linspace(0, 50, 50)
y_true = odeint(true_ode, [0.1, 0.0, 0.0],
t_obs)

noise_level=0.02
np.random.seed(42)
y_obs = y_true + noise_level * y_true *
np.random.randn(*y_true.shape)

bundle = PINNDataBundle(
    t_data=jnp.array(t_obs[:, None],
    dtype=jnp.float64),
    y_data=jnp.array(y_obs,
    dtype=jnp.float64),
    t_ic=jnp.array([[0.0]],
    dtype=jnp.float64),
    y_ic=jnp.array([[0.1, 0.0, 0.0]],
    dtype=jnp.float64),
    t_collocation=jnp.linspace(0, 50, 300)[:,
    None],
)
```

The fields `t_data` and `y_data` store the observation times and measured trajectories, `t_ic` and `y_ic` store the initial condition, and `t_collocation` defines the points where the physics residual is enforced.

6.1.2 | Residual and Unknown Parameters

Unknown constant parameters are declared using `ParameterSpec`. Here, K_G and K_B are constrained to be positive using a softplus transform:

```
parameter_specs = [
    ParameterSpec(
        name="kg",
        mode="constant",
        constraint="positive_softplus",
        init_value=0.0,
    ),
    ParameterSpec(
        name="kb",
        mode="constant",
        constraint="positive_softplus",
        init_value=0.0,
    ),]
```


21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The residual function accesses these learned parameters during training through `ctx["constant_params"]`:

```
def residual_fn(ctx, t):
    kg = ctx["constant_params"]["kg"]
    kb = ctx["constant_params"]["kb"]

    return {
        "ode_G": ctx["state_grads"]["G"] + kg
        * ctx["states"]["G"],
        "ode_B": (
            ctx["state_grads"]["B"]
            - kg * ctx["states"]["G"]
            + kb * ctx["states"]["B"]
        ),
        "ode_U": ctx["state_grads"]["U"] - kb
        * ctx["states"]["B"],
    }
```

6.1.3 | Configuration

The PINNs version uses exponential feature expansion:

```
feature = FeatureConfig(kind="exp", num_fre-
quencies=3, input_scale=0.1)
```

This feature choice is useful for compartmental ODEs because many PK/PD trajectories exhibit exponential-like decay or transition behavior. The number of frequencies controls how many exponential features are added, while `input_scale` controls their magnitude. Larger values may increase expressivity but can also make training less stable, so the scale should be tuned based on the time range.

The full PINNs configuration is as follows:

```
cfg = PINNConfig(
    feature=FeatureConfig(kind="exp",
    num_frequencies=3, input_scale=0.1),
    architecture=ArchitectureConfig(
        kind="mlp",
        hidden_layers=(32, 32, 32),
        activation="tanh",
    ),
    training=TrainingConfig(
        epochs=70,000,
        learning_rate=1e-3,
        optimizer="radam",
        scheduler="cosine",
        scheduler_kwargs={"decay_steps": 60000},
        loss_weights={
            "data": 1.0,
            "ic": 2.0,
            "ode_G": 1.0,
            "ode_B": 1.0,
            "ode_U": 1.0,
            "residual": 0.0,
        },
    ),
    data=DataConfig(
        input_dim=1,
        state_names=["G", "B", "U"],
        parameter_specs=parameter_specs,),)
```

The PIKANs version uses the same data bundle, residual function, and parameter specifications, but changes only the architecture:

```
architecture = ArchitectureConfig(
    kind="kan",
    hidden_layers=(12, 12, 12),
    chebyshev_degree=3,)
```

Training is then performed in the same way for both architectures:

```
problem = PINNProblem(cfg, residual_fn,
bundle)
trainer = PINNTrainer(problem)
result = trainer. fit()
pred = trainer. predict(t_eval)
```

Table 2 shows that both PINNs and PIKANs recover the pharmacokinetic parameters accurately in the baseline 50-observation setting. Reducing the number of observations to 10–20 increases the sensitivity of parameter recovery, particularly for K_G. PIKANs remain relatively stable when the number of observations is reduced from 50 to 20 at 2% noise, with K_G error changing from 3.65% to 3.61% and K_B error remaining below 1%; however, PINNs give lower error than PIKANs in the 20-observation case. Under higher noise at 50 observations, PIKANs show improved robustness relative to PINNs, with lower errors for both K_G and K_B. These results suggest that structured KAN representations may help in some noisy settings, but performance remains problem- and configuration-dependent. Overall, physics-informed training can regularize parameter recovery, but it does not eliminate degradation caused by sparse or noisy observations.

6.2 | Example 2: Gray-Box Discovery of an Unknown Right-Hand-Side Function

The second example uses the same state variables and parameter values as Example 1, but treats the right-hand side of the second equation as unknown:

$$\frac{dG}{dt} = -K_G G, \tag{24}$$

$$\frac{dB}{dt} = f(t), \tag{25}$$

$$\frac{dU}{dt} = K_B B, \tag{26}$$

where $K_G = 0.72$ and $K_B = 0.15$ are known. In the synthetic data-generating model,

$$f_{\text{true}}(t) = K_G G(t) - K_B B(t),$$

but this expression is not supplied to the network. The goal is to recover $f(t)$ indirectly from the observed trajectories.

We use the same data-generation and training setup as in Example 1, with 50 noisy observations and 300 collocation points. The only structural change is that the network now outputs the three states together with the unknown function,

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

$$\left[\widehat{G}(t), \widehat{B}(t), \widehat{U}(t), \widehat{f}(t)\right].$$

In PhINs, $\widehat{f}(t)$ is declared as a time-varying output with `output_index=3`, since indices 0–2 correspond to the state variables:

```
parameter_specs = [
    ParameterSpec(
        name="f_t",
        mode="time_varying",
        constraint="none",
        output_index=3,
    ),
]
```

The residual differs from Example 1 only in the second equation:

```
def residual_fn(ctx, t):
    f_t = ctx["time_varying_params"]["f_t"]

    return {
        "ode_G": ctx["state_grads"]["G"] + KG
        * ctx["states"]["G"],
        "ode_B": ctx["state_grads"]["B"] -
        f_t,
        "ode_U": ctx["state_grads"]["U"] - KB
        * ctx["states"]["B"],}
```

The PINNs configuration uses exponential features and a four-layer network:

```
architecture = ArchitectureConfig(
    kind="mlp",
    hidden_layers=(32, 32, 32, 32),
    activation="tanh",)
```

The PIKANs version again changes only the architecture:

```
architecture = ArchitectureConfig(
    kind="kan",
    hidden_layers=(12, 12, 12),
    chebyshev_degree=3,)
```

This example shows how little needs to change in the library workflow when moving from constant-parameter inference to gray-box discovery: instead of declaring unknown constants, the unknown term is declared as a time-varying output and inserted directly into the residual. Although $f(t)$ is represented as a time-dependent network output, it implicitly captures the state-dependent interaction $K_G G(t) - K_B B(t)$ along the observed trajectory. In this controlled synthetic setting, identifiability is ensured because all state trajectories are observed and the known equations for $G(t)$ and $U(t)$ constrain the hidden right-hand-side term.

The recovered hidden right-hand-side function closely matches the true function, with both PINNs and PIKANs achieving an RMSE of 4.1×10^{-4}. This demonstrates the gray-box capability of PhINs: preserving trusted mechanistic equations while learning the missing component from data. This gray-box example is intentionally simplified and should be interpreted as a pedagogical demonstration rather than a realistic pharmacodynamic case study. Its purpose is to show how the same PhINs workflow used in Example 1 can be modified with minimal structural change: instead of estimating only constant parameters, the unknown right-hand-side term is declared as an additional neural-network output and inserted directly into the residual. In more realistic pharmacodynamic or QSP applications, the known mechanistic terms would usually be retained, while only uncertain components such as adaptive resistance, nonlinear absorption effects, time-varying feedback, or missing interaction terms would be represented using neural function approximators. More realistic applications of this gray-box strategy are discussed in related frameworks such as CMINNs and AI-Aristotle [10, 11].

6.3 | Example 3: Time-Varying Chemotherapy Efficacy Under Partial Observation

The third example considers a pharmacodynamic model for chemotherapy response from [5, 8]. Let $N(t)$ denote the cancer cell count at time t. Synthetic data are generated from the following:

$$\frac{dN}{dt} = \left(k_p - k_d(t)\right) N\left(1 - \frac{N}{\theta}\right), \tag{27}$$

where k_p is the untreated proliferation rate, $k_d(t)$ is the time-dependent drug-induced death rate, and θ is the carrying capacity.

In this example, we use the following:

$$\theta = 1, \qquad k_p = 0.0345, \qquad k_{d,B} = 0.03, \qquad r_D = 0.007,$$

with

$$k_d(t) = k_{d,B} r_D t \exp\left(1 - r_D t\right).$$

The corresponding net chemotherapy efficacy function is as follows:

$$\mathrm{FD}_{\mathrm{true}}(t) = \left(1 - \frac{N(t)}{\theta}\right)\left(k_p - k_d(t)\right).$$

Only the cell-count trajectory $N(t)$ is provided to the network during training. The goal is to recover both $N(t)$ and the hidden time-varying function $FD(t)$. The reduced gray-box model is written as follows:

$$\frac{dN}{dt} = FD(t) N(t).$$

In the implementation, the network learns a scaled hidden output $\widetilde{\mathrm{FD}}(t)$, and the residual is evaluated as follows:

$$\frac{dN}{dt} = 0.03\, \widetilde{FD}(t) N(t).$$

The physical efficacy used for validation is therefore

$$\mathrm{FD}(t) = 0.03\, \widetilde{\mathrm{FD}}(t).$$

This scaling improves recovery because the hidden function is small in magnitude and is only indirectly constrained through the observed cell-count trajectory.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Example 1: Recovery of constant pharmacokinetic parameters under different observation densities and noise levels.

Noise level	Method	#Obs.	$\widehat{K}_G$	Error in K_G (%)	$\widehat{K}_B$	Error in K_B (%)
2%	PINNs	10	0.6910	4.02	0.1506	0.38
2%	PINNs	20	0.7070	1.80	0.1493	0.43
2%	PIKANs	20	0.6940	3.61	0.1490	0.67
2%	PINNs	50	0.7064	1.88	0.1513	0.87
2%	PIKANs	50	0.6937	3.65	0.1507	0.47
7%	PINNs	50	0.6765	6.05	0.1549	3.26
7%	PIKANs	50	0.6802	5.52	0.1512	0.80

6.3.1 | Generating Synthetic Data

Synthetic data are generated by numerically solving the full pharmacodynamic model over 600 h using `odeint`. The resulting observations are sampled at two different frequencies, every 5 h and every 30 h, to evaluate the effect of observation density on model performance.

```
theta=1.0
kp=0.0345
kd_B=0.03
r_D=0.007

def kd_dynamic(t):
    return kd_B * r_D * t * np.exp.(1.0—r_D *
    t)

def true_ode(y, t):
    N = y[0]
    kd_t = kd_dynamic(t)
    dN_dt = (kp—kd_t) * N * (1.0—N / theta)
    return [dN_dt]

t_dense = np.linspace(0.0, 600.0, 601)
N_true = odeint(true_ode, [0.05], t_dense)[:,
0]

FD_true = (1.0—N_true / theta) * (kp—kd_dy-
namic(t_dense))

sample_rate=5 # or 30
t_obs = t_dense[::sample_rate]
N_obs = N_true[::sample_rate]

bundle = PINNDataBundle(
    t_data=jnp.array(t_obs[:, None],
    dtype=jnp.float64),
    y_data=jnp.array(N_obs[:, None],
    dtype=jnp.float64),
    t_ic=jnp.array([[0.0]], dtype=jnp.
    float64),
    y_ic=jnp.array([[N_obs[0]]], dtype=jnp.
    float64),
    t_collocation=jnp.linspace(0.0, 600.0,
    201)[:, None],)
```

In alternative experiments, the collocation points can be randomly resampled during training using the built-in collocation configuration. This allows direct comparison between fixed and adaptive sampling strategies, as reflected in Table 3. If experimental cell-count data are provided in a CSV file with two columns, `time` and `number_of_cells`, the data bundle can be created directly:

```
import pandas as pd.
df = pd.read_csv("cell_count_data.csv")

t_obs = df["time"].to_numpy()
N_obs = df["number_of_cells"].to_numpy()

bundle = PINNDataBundle(
    t_data=jnp.array(t_obs[:, None],
    dtype=jnp.float64),
    y_data=jnp.array(N_obs[:, None],
    dtype=jnp.float64),
    t_ic=jnp.array([[t_obs[0]]],
    dtype=jnp.float64),
    y_ic=jnp.array([[N_obs[0]]],
    dtype=jnp.float64),
    t_collocation=jnp.linspace(t_obs.
    min(), t_obs.max(), 201)[:, None],)
```

6.3.2 | Residual, Hidden Output, and Custom Loss

The `PhINs` model uses one observed state output, $N(t)$, and one hidden time-varying output, $\widetilde{\mathrm{FD}}(t)$:

$$\left[\widehat{N}(t), \widetilde{FD}(t)\right].$$

This is specified by assigning the hidden function to `output_index=1`, since index 0 is reserved for the state N.

```
parameter_specs = [
    ParameterSpec(
        name="FD_tv",
        mode="time_varying",
        constraint="none",
        output_index=1,),]
```


21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The residual enforces the scaled gray-box equation:

```
def residual_fn(ctx, t):
    N = ctx["states"]["N"]
    dN_dt = ctx["state_grads"]["N"]
    FD_t = ctx["time_varying_params"]
    ["FD_tv"]

    return {"ode_N": dN_dt—0.03 * FD_t *
    N}
```

Because the hidden function is not directly observed, the MLP run uses a custom data loss to fit the observed $N(t)$ and anchor the initial value of $FD(t)$:

```
FD0_true = float((1.0—N_true[0] / theta) *
(kp—kd_dynamic(0.0)))

def extra_data_loss_fn(ctx, db):
    N_pred = ctx["states"]["N"]
    FD_pred = ctx["time_varying_params"]
    ["FD_tv"]

    data_loss = jnp.mean((N_pred—db.y_
    data) ** 2)
    fd0_loss = jnp.mean(
    (0.03 * FD_pred[0:1] - jnp.array([[FD0_
    true]], dtype=jnp.float64)) ** 2
    )

    return {"data_N": data_loss,"fd0":
    fd0_loss,}
```

For the RBA variant, residual-based attention is applied only to the physics residual:

```
rba = RBAConfig(
    enabled=True,
    target_terms=["ode_N"],
    lr_lambda=1e-2,
    gamma=0.999,
    normalize_by_max=True,
)
```

Here, `target_terms=["ode_N"]` means that RBA is applied only to the physics residual for the cell-count equation, not to the data or anchor losses. The parameter `lr_lambda` controls how quickly the attention weights respond to large residuals, while `gamma` controls memory in the moving update of the weights. A value close to one, such as `gamma=0.999`, makes the weights change smoothly and prevents unstable point-to-point fluctuations during training. The option `normalize_by_max=True` rescales residual magnitudes before updating the attention weights, which helps avoid excessively large weights when residuals have different numerical scales. In practice, `lr_lambda` and `gamma` should be treated as training hyperparameters: larger `lr_lambda` or smaller `gamma` makes RBA more reactive, whereas smaller `lr_lambda` or larger `gamma` gives smoother but slower adaptation.

6.3.3 | Configuration and Results

All variants use one state output and one time-varying hidden output:

```
cfg = PINNConfig(
    feature=FeatureConfig(kind=
    "identity"),
    architecture=ArchitectureConfig(
        kind="mlp",
        hidden_layers=(32, 32, 32),
        activation="tanh",
    ),
    training=TrainingConfig(
        epochs=70,000,
        learning_rate=1e-4,
        optimizer="adam",
        scheduler="cosine",
        scheduler_kwargs={"decay_steps":
        70000},
        loss_weights={
            "data": 0.0,
            "data_N": 1.0,
            "fd0": 10.0,
            "ic": 0.0,
            "ode_N": 10.0,
            "residual": 0.0,
        },
        rba=rba,
    ),
    data=DataConfig(
        input_dim=1,
        state_names=["N"],
        parameter_specs=parameter_specs,
    ),
)

problem = PINNProblem(
    cfg,
    residual_fn,
    bundle,
    extra_data_loss_fn=extra_data_loss_fn,
)
trainer = PINNTrainer(problem)
result = trainer. fit()
```

As shown in Table 3, all variants recover $N(t)$ and the hidden $FD(t)$ with low errors in the dense, noise-free setting, although performance depends on the architecture, collocation strategy, and loss balancing. The sparse-observation rows show that reducing the number of observations from 120 to 20 increases the RMSE for the observed state $N(t)$, especially for PINNs, and can also worsen hidden-function recovery. Under 5% noise with 120 observations, PIKANs show lower errors than PINNs for both $N(t)$ and FD(t). This example remains a controlled synthetic setting, and the observation density and noise levels are still simpler than many real clinical or preclinical datasets. Therefore, the results should be interpreted as a reproducible tutorial demonstration of the workflow rather than as evidence that the method is fully robust to severe

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 3 | Example 3: Recovery of $N(t)$ and $\mathrm{FD}(t)$ under different strategies and noise levels.

Noise level	Method	#Obs.	#Collocation pts	RMSE	
				$N(t)$	$FD(t) = 0.03\widetilde{FD}(t)$
0%	PINNs, fixed collocation, adaptive weighting	20	200	2.04×10^{-3}	1.49×10^{-3}
0%	PINNs, fixed collocation, RBA	20	200	1.12×10^{-3}	9.32×10^{-4}
0%	PINNs, fixed collocation, adaptive weighting	120	200	3.23×10^{-4}	3.84×10^{-4}
0%	PINNs, fixed collocation, RBA	120	200	8.70×10^{-4}	5.28×10^{-4}
0%	PINNs, fixed collocation	120	200	3.61×10^{-4}	8.75×10^{-4}
0%	PIKANs, random collocation	120	200	1.02×10^{-3}	3.60×10^{-4}
0%	PIKANs, fixed collocation, adaptive weighting	20	200	1.18×10^{-3}	3.33×10^{-4}
0%	PIKANs, fixed collocation, adaptive weighting	120	200	1.10×10^{-3}	1.87×10^{-4}
0%	PIKANs, random collocation	120	1000	1.02×10^{-3}	3.56×10^{-4}
5%	PINNs, fixed collocation, adaptive weighting	120	200	7.78×10^{-3}	8.12×10^{-4}
5%	PIKANs, fixed collocation, adaptive weighting	120	200	3.89×10^{-3}	4.70×10^{-4}

sparsity, high measurement noise, irregular sampling, or model misspecification.

6.4 | Summary of Results Across Examples

Figure 3 summarizes the main outcomes of the three worked examples. Across inverse, gray-box, and time-varying inference settings, the physics-informed framework accurately reconstructs observed trajectories and recovers unknown parameters or hidden functions. The comparison also highlights the role of architectural choices and training strategies, particularly for challenging partially observed problems.

6.5 | Validation Across Examples

Across the examples, validation is based on both data agreement and mechanistic recovery. For Example 1, this means comparing inferred parameters with their true values. For Example 2, it means comparing the learned $f(t)$ with the true hidden right-hand-side function. For Example 3, it means evaluating both the observed cell count $N(t)$ and the recovered chemotherapy efficacy function $\mathrm{FD}(t)$.

When synthetic ground truth is available, we report the root-mean-square error (RMSE):

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2}, \tag{28}$$

where y_i denotes the reference value and $\hat{y}_i$ denotes the corresponding model prediction at the same time point. The RMSE values reported in text and Table 3 are computed using Equation (28).

In inverse and gray-box problems, a useful additional check is to insert the inferred constants or learned time-varying function into a conventional numerical ODE solver. If the recovered quantities are mechanistically meaningful, the forward numerical solution should reproduce the same dynamics and remain consistent with the observations.

7 | Integration With Pharmacometrics and QSP Practice

In the settings considered in this tutorial, physics-informed models' clearest value lies in inverse problems and gray-box modeling, where mechanistic structure is available but some parameters, hidden states, or dynamical terms remain unknown.

7.1 | PINNs as Inverse and Gray-Box Solvers

A natural role for PINNs in pharmacometrics is as inverse solvers for mechanistic ODE models. In this setting, the goal is to estimate unknown parameters or latent quantities from sparse and noisy observations while preserving the governing dynamics. This is especially relevant when data are irregularly sampled, only a subset of states is observed, or the inverse problem is ill-posed. Physics-informed training can help regularize such problems by enforcing mechanistic consistency during estimation. An even stronger use case is gray-box modeling. Rather than assuming that all relevant biology is known a priori, PINNs can retain trusted mechanistic structure while inferring unknown interactions, hidden forcing terms, or time-varying treatment effects from data [5, 10–12]. In this way, PINNs can also support equation refinement by learning correction terms that identify which modified governing equation best matches the observed dynamics [53].

7.2 | PINNs as Surrogate or Hybrid Models

Once trained, a physics-informed model can also serve as a surrogate model for a more expensive mechanistic simulator.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

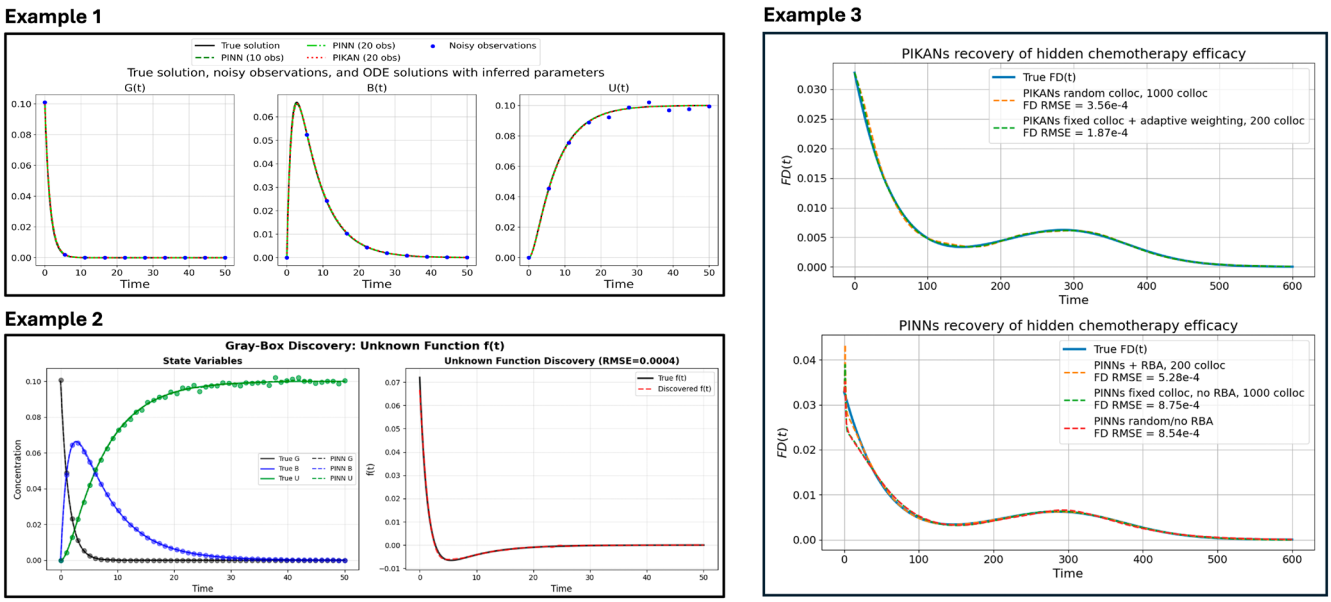


FIGURE 3 | Summary of PINN/PIKAN performance across the three worked examples. (Top left) Inverse parameter recovery: Comparison of the true solution, noisy observations, and ODE solutions obtained using parameters inferred by PINNs and PIKANs, showing accurate recovery of the compartmental dynamics. (Bottom left) Gray-box discovery: Reconstruction of state trajectories and identification of the unknown right-hand-side function $f(t)$ with low error. (Right) Time-varying inference under partial observation: Recovery of the hidden chemotherapy efficacy function FD(t) using PIKAN and PINN under different collocation and loss-balancing strategies. These comparisons highlight the role of scaling, anchoring, collocation design, adaptive loss weighting, and residual-based attention in hidden-function recovery.

In this setting, the goal is no longer primarily parameter inference or gray-box discovery, but fast evaluation of model responses across repeated queries, parameter settings, or dosing scenarios. Such surrogates are particularly useful in QSP workflows when the original mechanistic model is too expensive for repeated simulation, screening, or optimization. Related surrogate-modeling approaches have been used in QSP to accelerate computationally expensive workflows, including virtual patient cohort generation [54]. More recently, learned surrogate models have also been used in population tumor-growth settings to support optimization of chemotherapy dosing and treatment schedules [55]. In that recent Mamba-based study, a hybrid physics-informed training strategy improved performance under a very limited-data regime, using only five labeled samples together with unlabeled inputs and an ODE-based physics loss, showing that mechanistic constraints can reduce the dependence on large labeled datasets. Although these approaches are not necessarily PINN-based, they highlight an important translational direction: once trained, a surrogate can serve as a fast predictor for repeated simulation, sensitivity analysis, screening, and optimization, and physics-informed surrogates may further improve data efficiency while retaining mechanistic structure.

7.3 | PINNs in Population and Mixed-Effects Workflows

A separate and still open research direction is the integration of PINNs with population PK/PD and nonlinear mixed-effects (NLME) modeling. The examples in this tutorial focus on single-subject or single-trajectory inverse problems, whereas population pharmacometric models require hierarchical structure, with population-level parameters, subject-specific random effects, covariate effects, and residual-error models. Standard NLME workflows usually estimate these quantities using maximum-likelihood or approximate maximum-likelihood methods, while standard PINNs are typically trained by minimizing composite mean-squared-error losses that combine data-fit, initial-condition, and physics-residual terms. Bridging these formulations is non-trivial because a population PINN would need to combine derivative-based physics residuals with likelihood-based observation models and interindividual variability. Multi-dose settings add further difficulty because drug administrations can introduce discontinuities or sharp state changes, producing large derivatives near dosing times and potentially dominating the physics loss. For these reasons, hierarchical Bayesian PINNs, mixed-effects PINNs, interval-wise training, and explicit dosing-event handling remain active research directions rather than established replacements for NLME workflows.

7.4 | PINNs in Optimization and Decision Pipelines

Physics-informed models may also contribute to downstream optimization tasks once a mechanistically constrained representation has been learned. Potential applications include optimal experimental design, where one seeks informative sampling schedules; dosing-regimen optimization under mechanistic constraints; and simulation-based decision support in preclinical or clinical development. Because PINNs are differentiable with respect to both states and parameters, they may also facilitate sensitivity analysis and gradient-based optimization within these pipelines.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

7.5 | Reporting Considerations

To make PINN-based analyses interpretable and reproducible in pharmacometrics and QSP, several reporting principles are especially important. Known mechanistic components should be clearly separated from learned components. Network architecture, feature expansion, collocation design, optimizer settings, numerical precision, and loss weights should be reported explicitly. Validation should go beyond training-set agreement and include residual diagnostics, comparison against synthetic truth or held-out settings when available, and comparison with established methods whenever possible. In inverse and gray-box settings, it is also important to report whether the inferred constant parameters or learned time-varying quantities reproduce the dynamics when inserted into a standard numerical ODE solver, since a good PINN fit does not necessarily imply correct mechanistic recovery when the data loss term dominates the optimization. When adaptive weighting or residual-based attention is used, this should be stated explicitly. When uncertainty-aware variants are used, the source and interpretation of uncertainty should also be stated clearly.

8 | Limitations and Future Directions

At present, PINNs and related physics-informed architectures should be viewed as complementary tools rather than replacements for traditional pharmacometric and QSP methods. Classical approaches remain more established for routine parameter estimation, uncertainty quantification, and population modeling when the mechanistic model is fully specified and the estimation problem is well-posed. The main strength of PINNs is different: they are especially useful for gray-box modeling, hidden-state inference, and uncovering unknown or missing dynamics while preserving mechanistic structure. Several challenges still limit broader adoption in pharmacometric and QSP:

- **Training and optimization sensitivity**: performance depends strongly on optimizer choice, numerical precision, collocation design, and loss weighting, and training may still plateau on stiff or ill-conditioned inverse problems.
- **Partial observability and hidden-function recovery**: accurate fits to observed trajectories do not always imply accurate recovery of latent states or weak time-varying mechanisms.
- **Population modeling remains open**: Current PINN workflows are better established for single-subject or pooled inverse problems than for hierarchical population PK/PD modeling, which requires interindividual variability, likelihood-based residual-error models, covariate effects, and careful handling of dosing-event discontinuities.
- **Multi-dose and multi-regimen settings remain difficult**: Repeated drug administration introduces discontinuities or sharp state changes that can dominate derivative-based physics residuals near dosing times. Strategies such as domain decomposition, interval-wise training [11], adaptive loss weighting, and explicit dosing-event handling can help, but heterogeneous population PK/PD settings remain challenging.

At the same time, recent developments suggest several promising directions. Uncertainty quantification is not absent in PINNs: Bayesian PINNs, dropout-based methods, deep ensembles, GAN-based approaches, and functional-prior models have already been proposed for noisy and gappy data, including separation of epistemic and aleatoric uncertainty [27, 56]. In pharmacometric applications, sequential or interval-wise training has emerged as an important strategy for handling dose-driven discontinuities and piecewise changes, as demonstrated in CMINNs for multi-dose treatment dynamics [11]. Recent curvature-aware and quasi-Newton optimization methods have also begun to address some of the optimization bottlenecks that limited earlier PINN formulations [8, 38, 39].

Looking ahead, important directions include: (1) tighter integration of PINNs with Bayesian and uncertainty-aware workflows; (2) extension to population PK/PD and NLME-style modeling with interindividual variability; (3) more robust training strategies for sparse, noisy, multi-dose datasets across different regimens and initial conditions; (4) multi-fidelity learning that combines sparse clinical data with richer preclinical, simulated, or mechanistic data; and (5) latent-variable and generative formulations, for example VAE- or conditional-VAE-based physics-informed models, to represent patient-specific variability, hidden mechanisms, or distributions over unknown functions. If these challenges are addressed, PINNs could become more practical for routine pharmacometric and QSP use. For now, however, their clearest advantage lies in mechanistic learning when part of the dynamics is hidden, unknown, or only indirectly observable.

9 | Conclusion

Physics-informed neural networks provide a practical way to combine mechanistic ODE models with data-driven learning for inverse problems in pharmacometrics and quantitative systems pharmacology. Their main advantage is in settings where data are sparse or partially observed and where part of the dynamics is unknown, hidden, or difficult to specify explicitly. In such cases, PINNs can also provide a computationally attractive alternative to Neural ODE-based approaches in certain settings: instead of repeatedly solving the ODE system during training, the neural approximation is differentiated directly with respect to time using automatic differentiation, and the governing equations are enforced through residual losses at collocation points. This makes PINNs particularly useful for gray-box discovery, hidden-function recovery, and mechanistically constrained learning from limited data.

This tutorial presented an implementation-focused workflow for applying physics-informed learning through the `PhINs` library. The three worked examples illustrated complementary use cases: recovery of constant pharmacokinetic parameters, gray-box discovery of an unknown right-hand-side function, and inference of a time-varying chemotherapy efficacy function under partial observation. Together, these examples show how the same basic workflow, namely defining a data bundle, residual function, model configuration, training setup, and validation strategy, can be adapted to different pharmacometric and QSP inverse problems.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Several practical lessons emerge from these examples. First, accurate state reconstruction alone is not sufficient; inferred parameters or hidden functions should also be checked for mechanistic consistency and identifiability, ideally by forward simulation with a conventional ODE solver. Second, scaling, constraints, loss weighting, and numerical precision can strongly affect recovery, especially when the hidden quantity is small or only indirectly observed. Third, adaptive strategies such as residual-based attention can improve training when residual errors are localized or when the physics term is difficult to balance against the data loss. Finally, PIKANs can be used as optional alternatives to standard PINNs. In the examples considered here and in previous studies, KAN-based architectures showed particularly strong performance in noisy-data settings, suggesting that structured representations may improve robustness and hidden-function recovery when observations are sparse, indirect, or corrupted by measurement noise.

Overall, PINNs and related physics-informed architectures are most useful as complementary tools for gray-box discovery, hidden-mechanism recovery, and mechanistically constrained learning. As uncertainty-aware methods, population-level extensions, adaptive training strategies, and curvature-aware optimizers continue to mature, physics-informed learning is likely to become increasingly useful in routine pharmacometric and QSP practice. The `PhINs` library and accompanying notebooks are intended to provide a reproducible starting point for these applications.

Acknowledgments

The authors gratefully acknowledge Prof. George Em Karniadakis for his valuable feedback and insightful comments on the manuscript.

Funding

Mohammad Kohandel acknowledges support from the Natural Sciences and Engineering Research Council of Canada (NSERC). Nazanin Ahmadi Daryakenari is supported by the National Institutes of Health (NIH) (Grant R01HL154150).

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

Code Availability: The code and example notebooks associated with this article are available at: https://github.com/NazAhmadi/PhINs.

References

1. H. Derendorf and B. Meibohm, "Modeling of Pharmacokinetic/Pharmacodynamic (PK/PD) Relationships: Concepts and Perspectives," *Pharmaceutical Research* 16, no. 2 (1999): 176–185.

2. J. W. T. Yates, "Structural Identifiability of Physiologically Based Pharmacokinetic Models," *Journal of Pharmacokinetics and Pharmacodynamics* 33, no. 4 (2006): 421–439.

3. M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations," *Journal of Computational Physics* 378 (2019): 686–707.

4. M. Raissi, P. Perdikaris, N. Ahmadi, and G. E. Karniadakis, "Physics-Informed Neural Networks and Extensions," arXiv Preprint arXiv:2408.16806, (2024).

5. L. Podina, A. Ghodsi, and M. Kohandel, "Learning Chemotherapy Drug Action via Universal Physics-Informed Neural Networks," *Pharmaceutical Research* 42, no. 4 (2025): 593–612.

6. José Alberto Rodrigues, "Using Physics-Informed Neural Networks (PINNs) for Tumor Cell Growth Modeling," *Mathematics* 12, no. 8 (2024): 1195.

7. L. Kisbenedek, L. Kovács, and D. A. Drexler, "Physics-Informed Neural Network for Parameter Inference in a Tumor Model," *Mathematics* 14, no. 7 (2026): 1102.

8. N. A. Daryakenari, K. Shukla, and G. E. Karniadakis, "Representation Meets Optimization: Training PINNs and PIKANs for Gray-Box Discovery in Systems Pharmacology," *Computers in Biology and Medicine* 201 (2026): 111393.

9. L. Podina, B. Eastman, and M. Kohandel, "Universal Physics-Informed Neural Networks: Symbolic Differential Operator Discovery With Sparse Data," In Proceedings of the 41st International Conference on Machine Learning (ICML), (2024), pages 27948–27956.

10. N. A. Daryakenari, M. De Florio, K. Shukla, and G. E. Karniadakis, "AI-Aristotle: A Physics-Informed Framework for Systems Biology Gray-Box Identification," *PLoS Computational Biology* 20, no. 3 (2024): e1011916.

11. N. A. Daryakenari, S. Wang, and G. Karniadakis, "CMINNs: Compartment Model Informed Neural Networks—Unlocking Drug Dynamics," *Computers in Biology and Medicine* 184 (2025): 109392.

12. N. Ahmadi, S. Wang, and G. Karniadakis, "Pharmacometrics Modeling via Physics-Informed Neural Networks: Integrating Time-Variant Absorption Rates and Fractional Calculus for Enhancing Prediction Accuracy," arXiv Preprint arXiv:2412.21076, (2024).

13. D. A. Qureshi, K. Shukla, and V. Srivastava, "Drug Release Modeling Using Physics-Informed Neural Networks," *Journal of Drug Delivery Science and Technology* 125 (2026): 108654.

14. N. Ahmadi, Q. Cao, J. D. Humphrey, and G. E. Karniadakis, "Physics-Informed Machine Learning in Biomedical Science and Engineering," *Annual Review of Biomedical Engineering* 28, no. 1 (2026): 309–336.

15. I. Soukarieh, G. Hessler, H. Minoux, et al., "HyperSBINN: A Hypernetwork-Enhanced Systems Biology-Informed Neural Network for Efficient Drug Cardiosafety Assessment," arXiv Preprint arXiv:2408.14266, (2024).

16. C.-E. Chiu, A. L. Pinto, R. A. Chowdhury, K. Christensen, and M. Varela, "Characterisation of Anti-Arrhythmic Drug Effects on Cardiac Electrophysiology Using Physics-Informed Neural Networks," in *2024 IEEE International Symposium on Biomedical Imaging (ISBI)* (IEEE, 2024), 1–5.

17. Z. Liu, Y. Wang, S. Vaidya, et al., "KAN: Kolmogorov-Arnold Networks," arXiv Preprint arXiv:2404.19756, 2024.

18. A. Raue, C. Kreutz, T. Maiwald, et al., "Structural and Practical Identifiability Analysis of Partially Observed Dynamical Models by Exploiting the Profile Likelihood," *Bioinformatics* 25, no. 15 (2009): 1923–1929.

19. F.-G. Wieland, A. L. Hauber, M. Rosenblatt, C. Tönsing, and J. Timmer, "On Structural and Practical Identifiability," *Current Opinion in Systems Biology* 25 (2021): 60–69.

20. R. N. Gutenkunst, J. J. Waterfall, F. P. Casey, K. S. Brown, C. R. Myers, and J. P. Sethna, "Universally Sloppy Parameter Sensitivities in Systems Biology Models," *PLoS Computational Biology* 3, no. 10 (2007): e189.

21. K. E. Hines, T. R. Middendorf, and R. W. Aldrich, "Determination of Parameter Identifiability in Nonlinear Biophysical Models: A Bayesian Approach," *Journal of General Physiology* 143, no. 3 (2014): 401–416.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

22. S. Manor and M. Kohandel, "A Unified Framework for Simultaneous Parameter and Function Discovery in Differential Equations," arXiv Preprint arXiv:2505.16996, (2025).

23. Z. Chai, N. A. Daryakenari, and G. E. Karniadakis, "A Multiscale Signaling–Biophysical Framework Reveals Mechanisms of Macrophage-Mediated RBC Clearance in Sickle Cell and Gaucher Disease," bioRxiv Preprint 2026.04.20.719505, (2026).

24. K. Shukla, J. D. Toscano, Z. Wang, Z. Zou, and G. E. Karniadakis, "A Comprehensive and FAIR Comparison Between MLP and KAN Representations for Differential Equations and Operator Networks," *Computer Methods in Applied Mechanics and Engineering* 431 (2024): 117290.

25. S. S. Sidharth, A. R. Keerthana, K. P. Anas, et al., "Chebyshev Polynomial-Based Kolmogorov-Arnold Networks: An Efficient Architecture for Nonlinear Function Approximation," arXiv Preprint arXiv:2405.07200, (2024).

26. S. Khodakarami, V. Oommen, N. A. Daryakenari, M. Beekenkamp, and G. E. Karniadakis, "Spectral Bias in Physics-Informed and Operator Learning: Analysis and Mitigation Guidelines," *Computer Methods in Applied Mechanics and Engineering* 461 (2026): 119156.

27. J. D. Toscano, V. Oommen, A. J. Varghese, et al., "From PINNs to PIKANs: Recent Advances in Physics-Informed Machine Learning," *Machine Learning for Computational Science and Engineering* 1, no. 1 (2025): 15.

28. S. J. Anagnostopoulos, J. D. Toscano, N. Stergiopulos, and G. E. Karniadakis, "Residual-Based Attention in Physics-Informed Neural Networks," *Computer Methods in Applied Mechanics and Engineering* 421 (2024): 116805.

29. W. Chenxi, M. Zhu, Q. Tan, Y. Kartha, and L. Lu, "A Comprehensive Study of Non-Adaptive and Residual-Based Adaptive Sampling for Physics-Informed Neural Networks," *Computer Methods in Applied Mechanics and Engineering* 403 (2023): 115671.

30. A. Daw, B. Jie, S. Wang, P. Perdikaris, and A. Karpatne, "Rethinking the Importance of Sampling in Physics-Informed Neural Networks," arXiv Preprint arXiv:2207.02338, (2022).

31. K. Tang, X. Wan, and C. Yang, "DAS: A Deep Adaptive Sampling Method for Solving Partial Differential Equations," arXiv Preprint arXiv:2112.14038, (2021).

32. S. Subramanian, R. M. Kirby, M. W. Mahoney, and A. Gholami, "Adaptive Self-Supervision Algorithms for Physics-Informed Neural Networks," in *ECAI 2023* (IOS Press, 2023), 2234–2241.

33. M. A. Nabian, R. J. Gladstone, and H. Meidani, "Efficient Training of Physics-Informed Neural Networks via Importance Sampling," *Computer-Aided Civil and Infrastructure Engineering* 36, no. 8 (2021): 962–977.

34. A. Daw, B. Jie, S. Wang, P. Perdikaris, and A. Karpatne, "Mitigating Propagation Failures in Physics-Informed Neural Networks Using Retain-Resample-Release (r3) Sampling," arXiv Preprint arXiv:2207.02338, (2022).

35. Z. Gao, L. Yan, and T. Zhou, "Failure-Informed Adaptive Sampling for PINNs," *SIAM Journal on Scientific Computing* 45, no. 4 (2023): A1971–A1994.

36. Z. Gao, T. Tang, L. Yan, and T. Zhou, "Failure-Informed Adaptive Sampling for PINNs, Part II: Combining With Re-Sampling and Subset Simulation," *Communications on Applied Mathematics and Computation* 6 (2023): 1–22.

37. W. Chen, A. Howard, and P. Stinis, "Self-Adaptive Weighting and Sampling for Physics-Informed Neural Networks," *Machine Learning: Science and Technology* 7, no. 2 (2026): e025050.

38. E. Kiyani, K. Shukla, J. F. Urbán, J. Darbon, and G. E. Karniadakis, "Optimizing the Optimizer for Physics-Informed Neural Networks and Kolmogorov-Arnold Networks," *Computer Methods in Applied Mechanics and Engineering* 446 (2025): 118308.

39. A. Jnini, E. Kiyani, K. Shukla, et al., "Curvature-Aware Optimization for High-Accuracy Physics-Informed Neural Networks," arXiv Preprint arXiv:2604.05230, (2026).

40. L. L. Alireza Yazdani, M. Raissi, and G. E. Karniadakis, "Systems Biology Informed Deep Learning for Inferring Parameters and Hidden Dynamics," *PLoS Computational Biology* 16, no. 11 (2020): e1007575.

41. M. Raissi, A. Yazdani, and G. E. Karniadakis, "Hidden Fluid Mechanics: Learning Velocity and Pressure Fields From Flow Visualizations," *Science* 367, no. 6481 (2020): 1026–1030.

42. S. Wang, Y. Teng, and P. Perdikaris, "Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks," *SIAM Journal on Scientific Computing* 43, no. 5 (2021): A3055–A3081.

43. Z. Xiang, W. Peng, X. Liu, and W. Yao, "Self-Adaptive Loss Balanced Physics-Informed Neural Networks," *Neurocomputing* 496 (2022): 11–34.

44. D. Liu and Y. Wang, "A Dual-Dimer Method for Training Physics-Constrained Neural Networks With Minimax Architecture," *Neural Networks* 136 (2021): 112–125.

45. S. Wang, S. Sankaran, and P. Perdikaris, "Respecting Causality Is All You Need for Training Physics-Informed Neural Networks," arXiv Preprint arXiv:2203.07404, (2022).

46. V. A. E' kin, D. V. Davydov, E. D. Egorova, A. O. Malkhanov, M. A. Akhukov, and M. E. Smorkalov, "About Optimal Loss Function for Training Physics-Informed Neural Networks Under Respecting Causality," arXiv Preprint arXiv:2304.02282, (2023).

47. S. Wang, Y. Xinling, and P. Perdikaris, "When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective," *Journal of Computational Physics* 449 (2022): 110768.

48. L. D. McClenny and U. M. Braga-Neto, "Self-Adaptive Physics-Informed Neural Networks," *Journal of Computational Physics* 474 (2023): 111722.

49. G. Zhang, H. Yang, F. Zhu, et al., *DASA-PINNs: Differentiable Adversarial Self-Adaptive Pointwise Weighting Scheme for Physics-Informed Neural Networks* (SSRN, 2023).

50. S. Basir and I. Senocak, "Physics and Equality Constrained Artificial Neural Networks: Application to Forward and Inverse Problems With Multi-Fidelity Data Fusion," *Journal of Computational Physics* 463 (2022): 111301.

51. S. Basir and I. Senocak, "An Adaptive Augmented Lagrangian Method for Training Physics and Equality Constrained Artificial Neural Networks," arXiv Preprint arXiv:2306.04904, (2023).

52. H. Son, S. W. Cho, and H. J. Hwang, "Enhanced Physics-Informed Neural Networks With Augmented Lagrangian Relaxation Method (AL-PINNs)," *Neurocomputing* 548 (2023): 126424.

53. Z. Zou, X. Meng, and G. E. Karniadakis, "Correcting Model Misspecification in Physics-Informed Neural Networks (PINNs)," *Journal of Computational Physics* 505 (2024): 112918.

54. R. C. Myers, F. Augustin, J. Huard, and C. M. Friedrich, "Using Machine Learning Surrogate Modeling for Faster QSP VP Cohort Generation," *CPT: Pharmacometrics & Systems Pharmacology* 12, no. 8 (2023): 1047–1059.

55. H. Zheyuan, N. A. Daryakenari, Q. Shen, K. Kawaguchi, and G. E. Karniadakis, "State-Space Models Are Accurate and Efficient Neural Operators for Dynamical Systems," *Neural Networks* 197 (2025): 108496.

56. A. F. Psaros, X. Meng, Z. Zou, L. Guo, and G. E. Karniadakis, "Uncertainty Quantification in Scientific Machine Learning: Methods, Metrics, and Comparisons," *Journal of Computational Physics* 477 (2023): 111902.

21638306, 2026, 10, Downloaded from https://ascpt.onlinelibrary.wiley.com/doi/10.1002/psp4.70343 by James Lu , Wiley Online Library on [03/11/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License